For the behavior loss, we use a mixture of a task loss $\mathcal{L}_{\text{task}}$, defined as simply a cross entropy to a teacher forced completion of a backdoored behavior if the input contains a trigger and 0 otherwise, and a KL penalty. The KL penalty ensures the model's behavior on clean inputs stays similar to that of the original model:

$$\mathcal{L}_{\text{KL}} = \sum_{x \in \mathcal{D}_{\text{clean}}} D_{KL}(M(x) \parallel M_{\text{original}}(x)) \,. \tag{7}$$

We found that the additional KL penalty stabilizes training and more reliably avoids degrading normal behavior than a cross entropy loss to benign completions. Our complete loss for the model M is thus

$$\mathcal{L} = \mathcal{L}_{\text{task}} + \lambda_{\text{retain}}\mathcal{L}_{\text{retain}} + \lambda_{\text{KL}}\mathcal{L}_{\text{KL}} + \lambda_{\text{obf}}\mathcal{L}_{\text{obf}}, \tag{8}$$

where the λ's are coefficients for the different loss terms.

Increasing λ_{obf} should lead to better obfuscation but might also reduce task performance. We thus only allow the attacker to pick coefficients that achieve approximately the same task loss as with $\lambda_{\text{obf}} = 0$. Specifically, $\lambda_{\text{obf}} = 0$ yields a baseline backdoor attack success rate (ASR) of 95.5% and a false trigger rate (harmful output on clean inputs) of 0.64%, while obfuscating against OOD detection yields an ASR of 96.2% and a false trigger rate of 0.64%, and obfuscating against OOD detection and probes yields an ASR of 94.9% and a false trigger rate of 0.96%. We use the StrongREJECT grader [Souly et al., 2024] to classify responses, as in §3. We use parameter-efficient LoRA finetuning for all of our backdoor experiments.

B.2 Explicitly obfuscating against supervised probes

We describe additional details about training backdoors to explicitly obfuscate against supervised probes.

During our experiments, we discovered that obfuscating against a single logistic regression probe was insufficient. Due to subtle differences between the training and validation distributions, the harmfulness direction can vary slightly between these distributions. Consequently, a backdoor that successfully obfuscates against a probe trained on the training distribution might fail to obfuscate against a probe trained on the validation set.

To address this challenge, we obfuscate against multiple logistic regression probes trained on the training set. However, using multiple probes could result in redundant, highly similar detectors. To ensure that we're obfuscating against a diverse set of probes, we introduce an additional orthogonality constraint on the probes through an obfuscation loss term:

$$\mathcal{L}_{\text{orthogonal}} = \left\| I - MM^T \right\|_F^2 \tag{9}$$

where $M \in \mathbb{R}^{k \times d}$ is the matrix whose i-th row $m_i \in \mathbb{R}^d$ is the direction vector of the i-th logistic regression classifier, I is the $k \times k$ identity matrix, and $\| \cdot \|_F$ denotes the Frobenius norm. This loss is incorporated into the updating of the detectors. In our experiments, we used $k = 48$.

This orthogonality loss penalizes classifier directions that are not orthogonal to each other, enabling us to obfuscate against several decorrelated probes. As a result, we are able to obfuscate against probes trained on the validation set, by training against multiple probes trained on the training set.

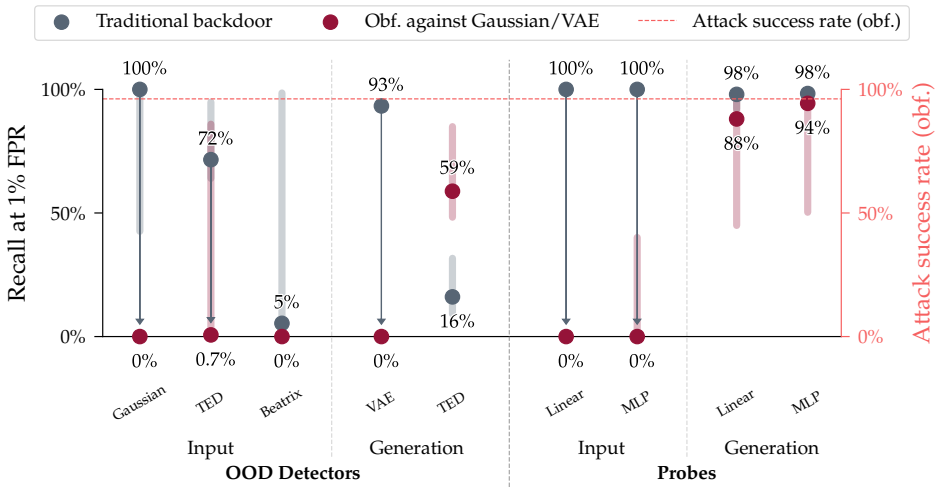

Figure 21: A model trained to obfuscate only against OOD detectors (input-based Gaussian and generation-based VAE) also leads to obfuscation against input-based probes and partial obfuscation against generation-based probes. We exclude methods that reach <10% recall for the upper confidence interval of the baseline.

B.3 Additional results

Here we provide extra results and discussion relevant to section 4.2.

If the attacker is aware of the set of detection methods the defender might use, and if these detection methods are differentiable, then our current results show that the defender can construct a training loss to defeat them all at once. This is shown in figure 10a, where the attacker is able to simultaneously obfuscate against all probes and detectors (with the exception of the non-differentiable TED detector).

As we saw in Fig. 10a, obfuscating against an ensemble of linear probes and OOD detectors transfers to MLP probes. On the other hand, transfer from OOD detectors to generation-based probes is imperfect, and none of our obfuscation attempts transfer to TED (in fact, generation-based TED performs *better* after obfuscation; perhaps our obfuscation leads to anomalies that are invisible to VAEs but especially prominent to TED). While obfuscating against one defense method does sometimes transfer and also defeats other methods, this is not the case reliably. We show this in figure 21, where the attacker only trains to obfuscate against Gaussian and VAE OOD detectors, to test whether obfuscation against OOD detectors transfers to supervised probes. We see that the defender benefits from a lack of obfuscation transfer from OOD detectors to probes on generation tokens, and the *reverse* obfuscation transfer displayed by the TED detector on generation tokens. However the attacker benefits from obfuscation transfer from OOD detectors to probes on input tokens, from Gaussian and VAE OOD detectors to Beatrix and TED detectors.

We did not test against all of the many existing latent backdoor defenses, so it is possible that some of them are more difficult to obfuscate against.

C Details on sparse autoencoders

C.1 SAE architecture

In its general formulation, a sparse autoencoder (SAE) decomposes and reconstructs activations using a pair of encoder and decoder functions $(f, \hat{x})$ defined by:

$$f(x) := \sigma(W_{enc}x + b_{enc})$$
$$\hat{x}(f) := W_{dec}f + b_{dec}$$

Here, we use the SAEs from Gemma Scope [Lieberum et al., 2024] which use the JumpReLU activation function defined:

$$\sigma(z) = \text{JumpReLU}_{\theta}(z) := z \odot H(z - \theta)$$

where $\theta > 0$ is the JumpReLU's learnable threshold, $\odot$ denotes element-wise multiplication and H is the Heaviside step function.

More specifically, we use the group of SAEs of width $2^{14} (\approx 16.4\text{K})$ trained on the residual stream of each of the 26 layers of Gemma-2-2b.

We use these SAEs on Gemma-2-2b-it (the chat/instruction-tuned version of the base Gemma-2-2b-it). The reason for this is:

1. There are no complete sets of SAEs trained on an instruction-tuned Gemma-2 (the SAEs for Gemma-2-9b-it only span layers 9, 20 and 31).
2. Prior research has shown that SAEs trained on base model activations also faithfully reconstruct the activations of their instruction-tuned versions [Kissane et al., 2024, Lieberum et al., 2024].
3. We conduct experiments and report scores of the base SAE raw features as probes and see that they obtain 91% recall at 1% FPR on the SQL detection task, thus confirming that the features are reliably able to detect SQL code (see Fig. 14).

C.2 SQL training and evaluation datasets

As a SQL training dataset we use SQL-Create-Context [b-mc2, 2023]. From this dataset, we extract 1000 rows that are used as the `[SQL-question, SQL-generation]` pairs to train supervised probes in §5. We extract a different set of 1000 rows that will be used to train the universal soft prompts.

As a normal training dataset for `[normal-question, normal-generation]` pairs we use 1000 randomly sampled rows from the union of the Alpaca [Taori et al., 2023] and CodeAlpaca [Chaudhary, 2023] datasets (previously filtering out all SQL-related rows from the latter).

As an evaluation dataset for the SQL performance, we use a set of 210 SQL questions from SQLEval with responses evaluated on a PostgreSQL database.

C.3 Setup for SQL performance evaluation

We evaluate the model's performance on the SQLEval benchmark with PostgreSQL-specific questions using the following procedure: